

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <assert.h>
4 #include <stdbool.h>
5
6 #include "linked_list.h"
7
8 node_t* new_node(int n) {
9     node_t* out = malloc(sizeof(node_t));
10    out->next = NULL;
11    out->val = n;
12    return out;
13 }
14
15 list_t* create_empty_list() {
16    list_t* linked = malloc(sizeof(list_t));
17    linked->head = NULL;
18    linked->tail = NULL;
19    linked->length = 0;
20    return linked;
21 }
22
23 list_t* create_example(int n) {
24    list_t* linked = create_empty_list();
25    for (int i = 0; i < n; i++) {
26        append(linked, i);
27    }
28    return linked;
29 }
30
31 list_t* create_single(int n) {
32    list_t* linked = create_empty_list();
33    push(linked, n);
34    return linked;
35 }
36
37 list_t* deep_copy(list_t* l) {
38    list_t* copy = create_empty_list();
39    node_t* to_append;
40    for (node_t* c = l->head; c != NULL; c = c->next) {
41        append(copy, c->val);
42    }
43
44    return copy;
45 }
46
47 bool is_empty(list_t* linked) {
48    return (linked->length == 0);
49 }
50
51 void show_list(list_t* linked) {
52    printf("TÊTE: %d\n", linked->head->val);
53    printf("QUEUE: %d\n", linked->tail->val);
54    printf("LONGUEUR: %d\n", linked->length);
55
56    node_t* curr = linked->head;
57    while (curr != NULL) {
58        printf("%d -> ", curr->val);
59        curr = curr->next;
60    }
```

```
61     printf("FIN\n");
62 }
63
64 void show_values(list_t* linked) {
65     for (node_t* c = linked->head; c != NULL; c = c->next) {
66         printf("%d -> ", c->val);
67     }
68     printf("T\n");
69 }
70
71 void push(list_t* linked, int n) {
72     node_t* new_head = malloc(sizeof(node_t));
73     new_head->val = n;
74
75     node_t* old_head = linked->head;
76     new_head->next = old_head;
77     linked->head = new_head;
78     linked->length += 1;
79
80     if (linked->length == 1) {
81         linked->tail = new_head;
82     }
83 }
84
85 void append(list_t* linked, int n) {
86     assert(linked != NULL);
87     if (linked->length == 0) {
88         node_t* new_tail = new_node(n);
89         linked->tail = new_tail;
90         linked->head = new_tail;
91         linked->length++;
92     } else {
93         node_t* new_tail = malloc(sizeof(node_t));
94         new_tail->val = n;
95         new_tail->next = NULL;
96
97         node_t* old_tail = linked->tail;
98         old_tail->next = new_tail;
99         linked->tail = new_tail;
100         linked->length += 1;
101     }
102 }
103
104 void insert(list_t* linked, int index, int x) {
105     assert(index <= linked->length);
106     if (index == 0) {
107         push(linked, x);
108     }
109     if (index == linked->length) {
110         append(linked, x);
111     }
112     else {
113         node_t* new_node = malloc(sizeof(node_t));
114         new_node->val = x;
115
116         node_t* curr = linked->head;
117         int i = 0;
118         while (i < index-1) {
119             curr = curr->next;
120             i += 1;
121         }
122         new_node->next = curr->next;
123         curr->next = new_node;
124         linked->length++;
125     }
126 }
```

```
121     }
122     new_node->next = curr->next;
123     curr->next = new_node;
124
125     linked->length += 1;
126 }
127 }
128
129 int pop(list_t* linked) {
130     assert(is_empty(linked) == false);
131     node_t* new_head = linked->head->next;
132     node_t* old_head = linked->head;
133     int val = old_head->val;
134     linked->head = new_head;
135     free(old_head);
136     return val;
137 }
138
139 node_t* free_get_next(node_t* node) {
140     assert(node != NULL);
141     node_t* next = node->next;
142     free(node);
143     return next;
144 }
145
146 void free_list(list_t* linked) {
147     assert(linked != NULL);
148     node_t* curr = linked->head;
149     while (curr != NULL) {
150         curr = free_get_next(curr);
151     }
152     free(linked);
153 }
154
```